

nopCommerce + OpenTelemetry

Observability for the Order Placement Flow

Software Architectures · Assignment 1

Ricardo Dias

The Problem

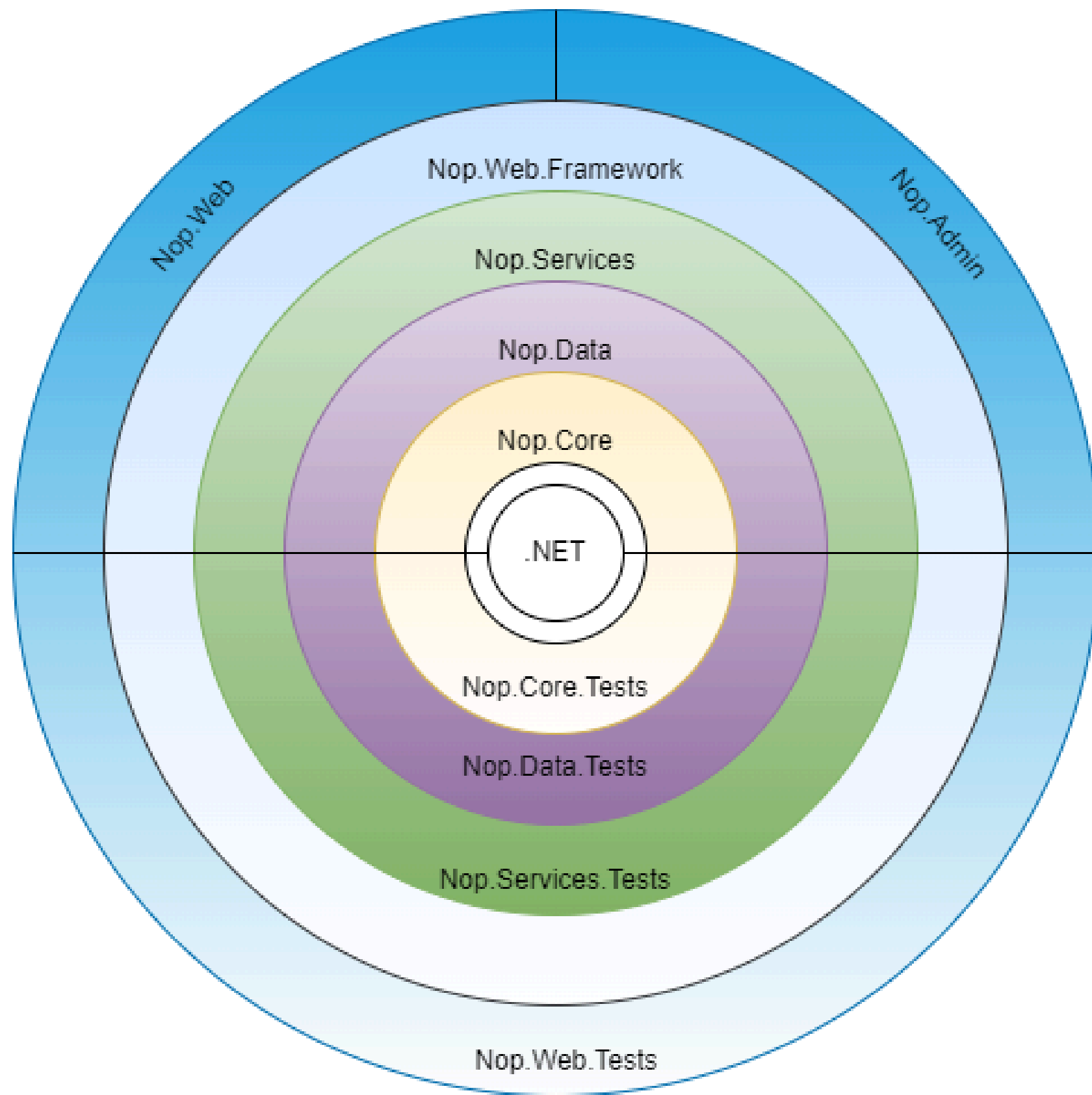
nopCommerce is an open-source, production-grade e-commerce platform built on ASP.NET Core. It powers real online stores with real checkout flows, payment processing, and customer data.

However, there is no way to observe the system at runtime.

- No distributed tracing
- No business-level metrics
- ...

Goal: Add OpenTelemetry tracing and metrics to the order placement flow, with a privacy strategy.

The Architecture



Onion Architecture

Dependencies only point inward (never outward). Outer layers can reference any inner layer directly.

IEventPublisher

In-process observer pattern. Consumers discovered via reflection at startup, called sequentially. A slow consumer blocks the publisher.

Dependency Rules

Enforced through .csproj project references. Nop.Core has no dependencies. Each outer layer adds references inward only.

Where Observability Is Easy vs Hard

Easy

ASP.NET Core middleware pipeline
Event system
Service-layer DI interfaces
Scheduled task runner
IHttpClientFactory

Hard

Custom ILogger writes to database
No distributed tracing infrastructure
Service locator pattern throughout
Swallowed exceptions (empty catch blocks)
Plugin isolation (31 plugins, no shared contract)

Why the Order Flow?

Flow	Risk if degraded
Product search	Frustrating
Customer places an order	Revenue loss + inconsistent payment state
Admin publishes product	Delayed catalog update

The checkout flow is the most operationally critical path.

Touches the most layers: Basket, Order, Payment, Inventory
Involves the main external dependency (payment gateway)

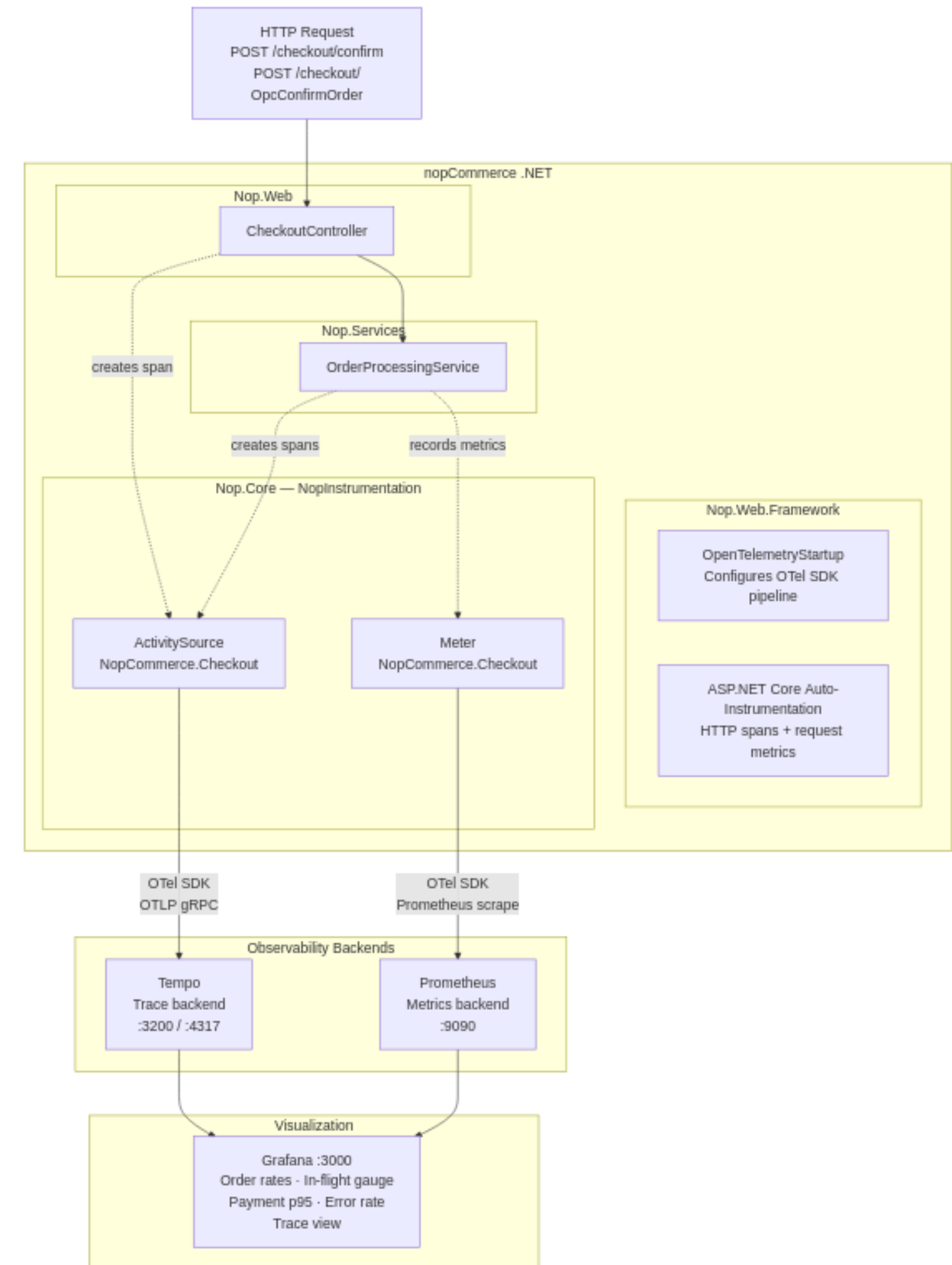
Instrumented Flow Architecture

The request flows from CheckoutController to OrderProcessingService, with spans at each critical stage:

Validation, payment, inventory...

Traces are exported via OTLP gRPC to Tempo; metrics are scraped by Prometheus. Both are visualised in Grafana

No existing interfaces or business logic modified



Distributed Tracing

Span	What it captures
checkout.confirm	Business-level entry point (child of HTTP span). Both checkout paths
order.place	Full order pipeline orchestration
order.validate_totals	Discount validation, tax calculation, totals
payment.process	Payment gateway call — main external dependency
order.save	Database write for the order record
inventory.adjust	Stock reduction per order item

Each span carries:

- Order ID as a tag for correlation
- Payment method system name, so operators can filter by gateway
- Success/failure status and sanitized error information

Privacy Strategy: What We Include vs Exclude

Rule: Span tags set explicitly with allowed fields only. No object serialization into span attributes.

Included (safe)	Excluded (PII / sensitive)
Order ID, product ID, category ID	Customer email, name, phone, IP address
Order completion status	Billing and shipping addresses
Failure reason category	Credit card numbers, CVV, expiry dates (even partially masked)
Payment method system name	ProcessPaymentRequest fields containing card data
Exception type name	Exception message or stack trace (may contain user data)
Discount codes in hashed/truncated form	Shopping cart contents (personal purchasing patterns)
Quantities, totals as aggregated numbers	Full error messages (may contain user-submitted data)

Privacy Strategy: Where Redaction Happens

PII-sensitive fields are never added in the first place. The pipeline is clean by construction.

Layer	Approach	Risk
Collector	Strip fields in the pipeline before storage	PII crosses the network before being stripped. A misconfigured pipeline leaks PII
Storage	Delete sensitive traces after a period	PII exists in transit, in collector memory, and in storage until deletion
Application (chosen)	Never emit PII from the application	Clean by construction. No misconfiguration can leak what was never sent

Metric 1: Orders Completed

Counter, tagged status=success/failure

"Are orders going through?"

There is no way to answer without querying the database. HTTP error rates are blind to business failures, such as insufficient stock.

This counter captures every checkout outcome at the business level, tagged success/failure.

Incremented at the end of `PlaceOrderAsync()` in a try/finally block.



Metric 2: Orders In Flight

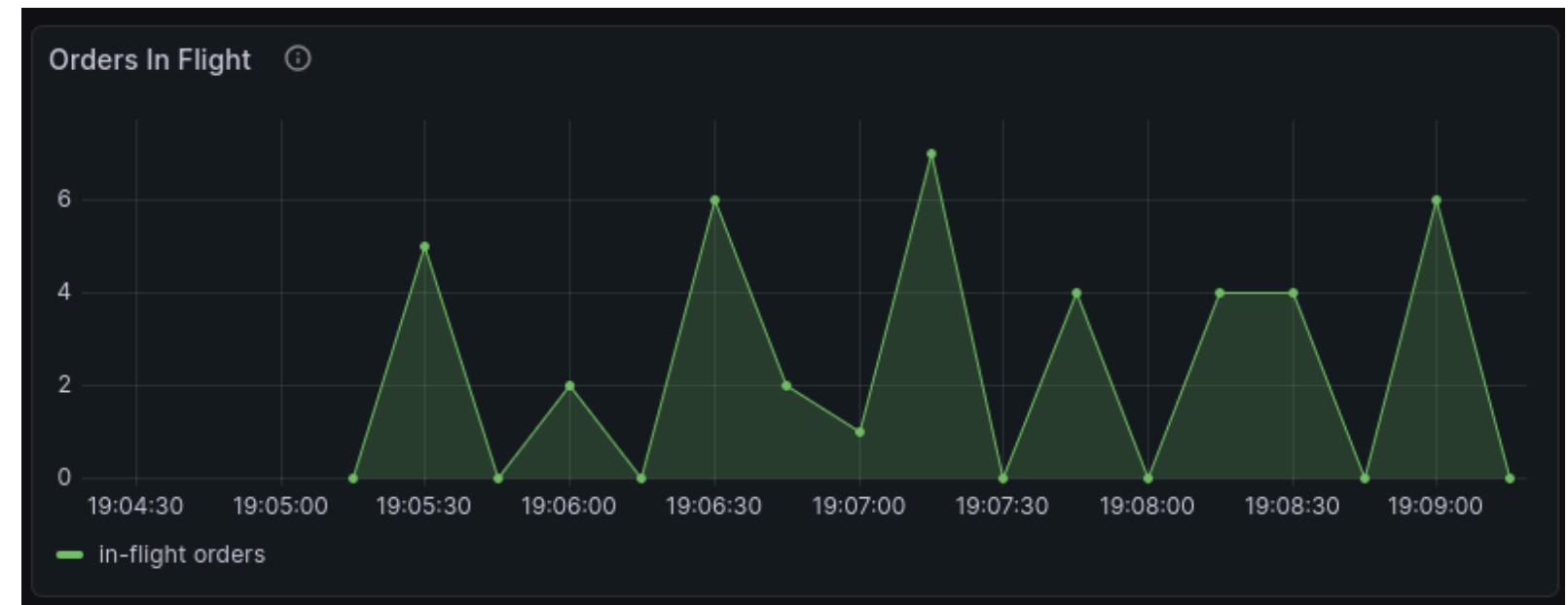
UpDownCounter

"Is the system saturating?"

If this value rises without falling, the system is accepting orders faster than it can complete them.

This counter tracks how many orders are currently inside the checkout pipeline. The aggregate value is the saturation signal.

Incremented at the start of `PlaceOrderAsync()`, decremented in a `finally` block.



Metric 3: Payment Gateway Latency

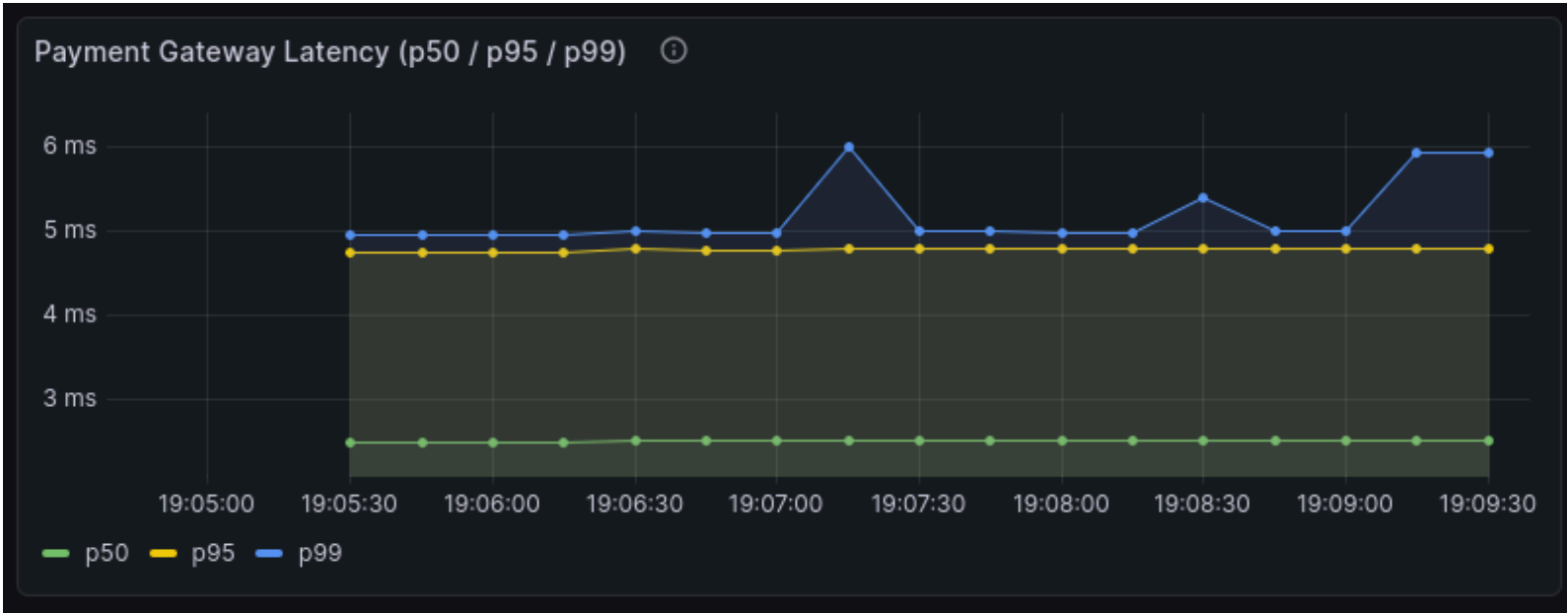
Histogram, tagged by payment method

"Is the payment provider degrading?"

The payment gateway is the most critical external dependency. Its latency can spike independently from internal performance, and plugins offer no shared observability contract.

This histogram records gateway call duration by payment method, making degradation visible.

Measured via Stopwatch in `GetProcessPaymentResultAsync()`, capturing only the gateway call.



Metric 4: Event Dispatch Duration

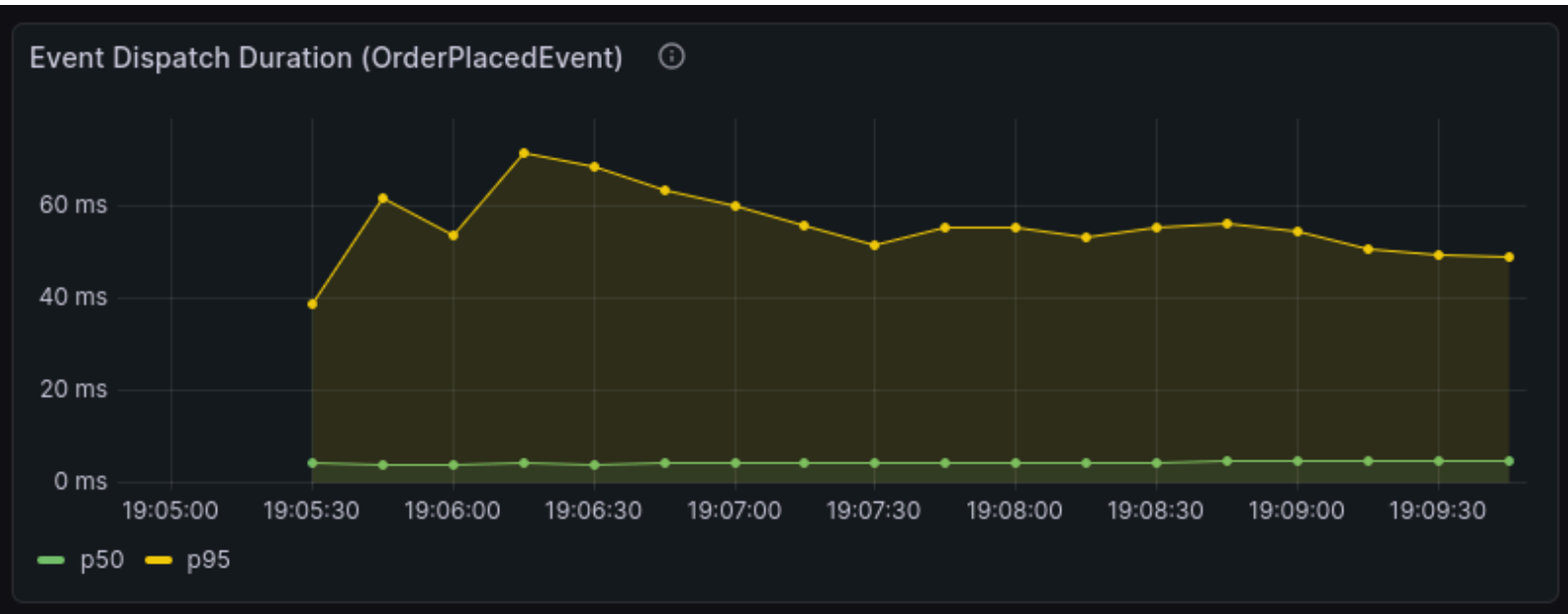
Histogram

"Is a consumer slowing down checkout?"

Event dispatch is sequential. Every consumer must complete before the customer gets a response. A slow consumer increases checkout latency with no visible cause.

This histogram records the total duration of event dispatch after order placement, surfacing that hidden cost.

Measured via Stopwatch around the PublishAsync call in PlaceOrderAsync().



DEMO